DevOps Cheat Sheet

Linux, Git, Docker, Kubernetes, Helm, Terraform

Every command, grouped by the tool and by what you are trying to do.

Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1 Linux **2**
1.1 Basic . . . 2
1.2 Intermediate . . . 2
1.3 Advanced . . . 3
1.4 Networking . . . 4
1.5 File management and search . . . 5
1.6 System monitoring . . . 5
1.7 Package management . . . 6
1.8 Disk and filesystem . . . 6
1.9 Scripting and automation . . . 6
1.10 Development and debugging . . . 7
1.11 Other useful commands . . . 7

2 Git **8**
2.1 Installation . . . 8
2.2 Basic . . . 8
2.3 Branching and merging . . . 9
2.4 Remote repositories . . . 9
2.5 Stashing and cleaning . . . 10
2.6 Tagging . . . 10
2.7 Advanced . . . 10
2.8 GitHub CLI . . . 10

3 Docker **12**
3.1 Installation . . . 12
3.2 Basic . . . 12
3.3 Intermediate . . . 12
3.4 Advanced . . . 13

4 Kubernetes **15**
4.1 Installation . . . 15
4.2 Basic . . . 15
4.3 Intermediate . . . 15
4.4 Advanced . . . 16

5 Helm **18**
5.1 Installation . . . 18
5.2 Basic . . . 18
5.3 Installing and upgrading charts . . . 18
5.4 Working with charts . . . 19
5.5 Inspecting and testing . . . 19
5.6 Values and customisation . . . 20
5.7 Namespaces and contexts . . . 20
5.8 Repositories and publishing . . . 21
5.9 History and rollbacks . . . 21

6 Terraform **22**
6.1 Installation . . . 22
6.2 Commands . . . 22

1. Linux

Linux is the ground every other tool here stands on. These commands navigate the filesystem, manage files, configure permissions and automate work from a terminal.

1.1 Basic

Command	Description	Syntax	Example
`pwd`	Print the current working directory	`pwd`	`pwd`
`ls`	List files and directories	`ls [options] [path]`	`ls -la /var/log`
`cd`	Change directory	`cd [path]`	`cd /etc/nginx`
`touch`	Create an empty file, or update its timestamp	`touch file...`	`touch notes.txt`
`mkdir`	Create a new directory	`mkdir [-p] dir...`	`mkdir -p src/api/v1`
`rm`	Remove files or directories	`rm [-rf] target...`	`rm -rf build/`
`rmdir`	Remove empty directories	`rmdir dir...`	`rmdir old-cache`
`cp`	Copy files or directories	`cp [-r] source dest`	`cp -r src/ backup/`
`mv`	Move or rename files and directories	`mv source dest`	`mv app.log app.log.1`
`cat`	Display the content of a file	`cat file...`	`cat /etc/hosts`
`echo`	Display a line of text	`echo [-n] text`	`echo "deploy done"`
`clear`	Clear the terminal screen	`clear`	`clear`

1.2 Intermediate

Command	Description	Syntax	Example
`chmod`	Change file permissions	`chmod [-R] mode file`	`chmod 755 deploy.sh`
`chown`	Change file ownership	`chown [-R] user:group file`	`chown -R app:app /srv/app`
`find`	Search for files and directories	`find path [tests] [action]`	`find . -name "*.log"`
`grep`	Search for text in a file	`grep [options] pattern file`	`grep -rn "TODO" src/`
`wc`	Count lines, words and characters in a file	`wc [-lwc] file`	`wc -l access.log`
`head`	Display the first few lines of a file	`head [-n N] file`	`head -n 20 app.log`
`tail`	Display the last few lines of a file	`tail [-f] [-n N] file`	`tail -f /var/log/syslog`
`sort`	Sort the contents of a file	`sort [options] file`	`sort -u hosts.txt`

Command	Description	Syntax	Example
`uniq`	Remove duplicate adjacent lines	`uniq [-c] [file]`	`sort ips.txt | uniq -c`
`diff`	Compare two files line by line	`diff [-u] file1 file2`	`diff -u old.conf new.conf`
`tar`	Archive files into a tarball	`tar -czf archive files`	`tar -czf site.tgz site/`
`zip/unzip`	Compress and extract ZIP files	`zip [-r] archive files`	`zip -r logs.zip logs/`
`df`	Display disk space usage	`df [-h] [path]`	`df -h /`
`du`	Display directory size	`du [-sh] path`	`du -sh /var/lib/docker`
`top`	Monitor system processes in real time	`top`	`top`
`ps`	Display active processes	`ps [options]`	`ps aux`
`kill`	Terminate a process by its PID	`kill [-SIGNAL] pid`	`kill -9 4821`
`ping`	Check network connectivity	`ping [-c N] host`	`ping -c 4 1.1.1.1`
`wget`	Download files from the internet	`wget [options] url`	`wget https://x.dev/f.tgz`
`curl`	Transfer data from or to a server	`curl [options] url`	`curl -fsSL https://x.dev/api`
`scp`	Securely copy files between systems	`scp [-r] source dest`	`scp app.jar srv:/opt/app/`
`rsync`	Synchronise files and directories	`rsync [options] src dest`	`rsync -avz site/ srv:/www/`

1.3 Advanced

Command	Description	Syntax	Example
`awk`	Text processing and pattern scanning	`awk 'program' file`	`awk '{print $1}' app.log`
`sed`	Stream editor for filtering and transforming text	`sed [-i] 'script' file`	`sed -i 's/8080/80/g' app.conf`
`cut`	Remove sections from each line of a file	`cut -d delim -f list file`	`cut -d: -f1 /etc/passwd`
`tr`	Translate or delete characters	`tr [options] set1 [set2]`	`tr 'a-z' 'A-Z' < in.txt`
`xargs`	Build and execute command lines from standard input	`xargs [options] command`	`find . -name "*.tmp" | xargs rm`
`ln`	Create symbolic or hard links	`ln [-s] target link`	`ln -s /opt/app/current app`
`df -h`	Display disk usage in human-readable format	`df -h [path]`	`df -h /var`

Command	Description	Syntax	Example
`free`	Display memory usage	`free [-h]`	`free -h`
`iostat`	Display CPU and I/O statistics	`iostat [interval] [count]`	`iostat 2 5`
`netstat`	Network statistics; `ss` is the modern alternative	`netstat [options]`	`netstat -tulpn`
`ifconfig/ip`	Configure network interfaces; `ip` is the modern alternative	`ip object command`	`ip link set eth0 up`
`iptables`	Configure firewall rules	`iptables [-t table] rule`	`iptables -L -n`
`systemctl`	Control the systemd system and service manager	`systemctl verb [unit]`	`systemctl restart nginx`
`journalctl`	View system logs	`journalctl [options]`	`journalctl -u nginx -f`
`crontab`	Schedule recurring tasks	`crontab [-e] [-l]`	`crontab -e`
`at`	Schedule tasks for a specific time	`at time`	`at 02:00 tomorrow`
`uptime`	Display system uptime	`uptime`	`uptime`
`whoami`	Display the current user	`whoami`	`whoami`
`users`	List all users currently logged in	`users`	`users`
`hostname`	Display or set the system hostname	`hostname [name]`	`hostname -f`
`env`	Display environment variables	`env [name=value] [cmd]`	`env | grep PATH`
`export`	Set environment variables	`export NAME=value`	`export PORT=8080`

1.4 Networking

Command	Description	Syntax	Example
`ip addr`	Display or configure IP addresses	`ip addr [show] [dev]`	`ip addr show eth0`
`ip route`	Show or manipulate routing tables	`ip route [show] [add]`	`ip route show default`
`traceroute`	Trace the route packets take to a host	`traceroute host`	`traceroute example.com`
`nslookup`	Query DNS records	`nslookup host [server]`	`nslookup example.com`
`dig`	Query DNS servers	`dig [@server] name [type]`	`dig example.com MX`
`ssh`	Connect to a remote server via SSH	`ssh [-p port] user@host`	`ssh -p 2222 app@srv1`

Command	Description	Syntax	Example
`ftp`	Transfer files using the FTP protocol	`ftp host`	`ftp files.example.com`
`nmap`	Network scanning and discovery	`nmap [options] target`	`nmap -sV 10.0.0.0/24`
`telnet`	Communicate with remote hosts	`telnet host [port]`	`telnet srv1 25`
`netcat (nc)`	Read and write data over networks	`nc [options] host port`	`nc -zv srv1 5672`

1.5 File management and search

Command	Description	Syntax	Example
`locate`	Find files quickly using a database	`locate pattern`	`locate nginx.conf`
`stat`	Display detailed information about a file	`stat file`	`stat app.log`
`tree`	Display directories as a tree	`tree [-L depth] [path]`	`tree -L 2 src/`
`file`	Determine a file's type	`file target`	`file build/app`
`basename`	Extract the filename from a path	`basename path [suffix]`	`basename /srv/app.jar`
`dirname`	Extract the directory part of a path	`dirname path`	`dirname /srv/app.jar`

1.6 System monitoring

Command	Description	Syntax	Example
`vmstat`	Display virtual memory statistics	`vmstat [interval] [count]`	`vmstat 1 5`
`htop`	Interactive process viewer, an alternative to `top`	`htop`	`htop`
`lsof`	List open files	`lsof [options]`	`lsof -i :8080`
`dmesg`	Print kernel ring buffer messages	`dmesg [options]`	`dmesg -T | tail`
`uptime`	Show how long the system has been running	`uptime`	`uptime`
`iotop`	Display real-time disk I/O by process	`iotop [options]`	`iotop -o`

1.7 Package management

Command	Description	Syntax	Example
`apt`	Package manager for Debian-based distributions	`apt verb [package]`	`apt install nginx`
`yum/dnf`	Package manager for RHEL-based distributions	`dnf verb [package]`	`dnf install nginx`
`snap`	Manage snap packages	`snap verb [package]`	`snap install docker`
`rpm`	Manage RPM packages	`rpm [-ivh] [-qa] [pkg]`	`rpm -qa | grep nginx`

1.8 Disk and filesystem

Command	Description	Syntax	Example
`mount/umount`	Mount or unmount filesystems	`mount device dir`	`mount /dev/sdb1 /mnt`
`fsck`	Check and repair filesystems	`fsck [-y] device`	`fsck -y /dev/sdb1`
`mkfs`	Create a new filesystem	`mkfs -t type device`	`mkfs -t ext4 /dev/sdb1`
`blkid`	Display information about block devices	`blkid [device]`	`blkid /dev/sdb1`
`lsblk`	List information about block devices	`lsblk [options]`	`lsblk -f`
`parted`	Manage partitions interactively	`parted device`	`parted /dev/sdb`

1.9 Scripting and automation

Command	Description	Syntax	Example
`bash`	Command interpreter and scripting shell	`bash [script] [args]`	`bash deploy.sh prod`
`sh`	Legacy shell interpreter	`sh [script]`	`sh install.sh`
`cron`	Automate recurring tasks, one job per crontab line	`m h dom mon dow cmd`	`0 3 * * * /opt/backup.sh`
`alias`	Create shortcuts for commands	`alias name='command'`	`alias gs='git status'`
`source`	Execute commands from a file in the current shell	`source file`	`source ~/.bashrc`

1.10 Development and debugging

Command	Description	Syntax	Example
`gcc`	Compile C programs	`gcc [options] file`	`gcc -o app main.c`
`make`	Build and manage projects	`make [target]`	`make build`
`strace`	Trace system calls and signals	`strace [-p pid] [cmd]`	`strace -p 4821`
`gdb`	Debug programs	`gdb [program] [core]`	`gdb ./app`
`git`	Version control system	`git subcommand [args]`	`git status`
`vim/nano`	Text editors for scripting and editing	`vim file`	`vim /etc/hosts`

1.11 Other useful commands

Command	Description	Syntax	Example
`date`	Display or set the system date and time	`date [+format]`	`date +%Y-%m-%d`
`cal`	Display a calendar	`cal [month] [year]`	`cal 8 2026`
`man`	Display the manual for a command	`man [section] command`	`man 5 crontab`
`history`	Show previously executed commands	`history [n]`	`history 20`

2. Git

Git tracks every change, lets a team work on the same code without collisions, and lets you undo a mistake. These commands cover the everyday work and the recovery tools you reach for when something has gone wrong.

2.1 Installation

```
# Windows
winget install Git.Git
winget install GitHub.cli
# macOS
brew install git gh
# Linux (Debian/Ubuntu) -- git is in the base repositories, gh is not
sudo apt install git
```

The GitHub CLI needs its own apt repository on Debian and Ubuntu:

```
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg | \
  sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg] \
  https://cli.github.com/packages stable main" | \
  sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update && sudo apt install gh
```

First run: `git` refuses to commit until it knows who you are, and `gh` needs a token before it can reach GitHub. Do both once per machine:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
gh auth login
```

Verify: `git --version` and `gh --version`. `gh auth status` says whether the token is still good.

2.2 Basic

Command	Description	Example
`git init`	Initialises a new Git repository in the current directory	`git init`
`git clone`	Copies a remote repository to the local machine	`git clone https://github.com/user/repo.git`
`git status`	Displays the state of the working directory and staging area	`git status`
`git add`	Adds changes to the staging area	`git add file.txt`
`git commit`	Records changes to the repository	`git commit -m "Initial commit"`
`git config`	Configures user settings, such as name and email	`git config --global user.name "Your Name"`

Command	Description	Example
`git log`	Shows the commit history	`git log`
`git show`	Displays detailed information about a specific commit	`git show <commit-hash>`
`git diff`	Shows changes between commits, the working directory and the staging area	`git diff`
`git reset`	Unstages changes or resets commits	`git reset HEAD file.txt`

2.3 Branching and merging

Command	Description	Example
`git branch`	Lists branches or creates a new branch	`git branch feature-branch`
`git checkout`	Switches between branches or restores files	`git checkout feature-branch`
`git switch`	Switches branches, the modern alternative to `git checkout`	`git switch feature-branch`
`git merge`	Combines changes from one branch into another	`git merge feature-branch`
`git rebase`	Moves or combines commits from one branch onto another	`git rebase main`
`git cherry-pick`	Applies specific commits from one branch to another	`git cherry-pick <commit-hash>`

2.4 Remote repositories

Command	Description	Example
`git remote`	Manages remote repository connections	`git remote add origin https://github.com/user/repo.git`
`git push`	Sends changes to a remote repository	`git push origin main`
`git pull`	Fetches and merges changes from a remote repository	`git pull origin main`
`git fetch`	Downloads changes from a remote repository without merging	`git fetch origin`
`git remote -v`	Lists the URLs of remote repositories	`git remote -v`

2.5 Stashing and cleaning

Command	Description	Example
`git stash`	Temporarily saves changes not yet committed	`git stash`
`git stash pop`	Applies stashed changes and removes them from the stash list	`git stash pop`
`git stash list`	Lists all stashes	`git stash list`
`git clean`	Removes untracked files from the working directory	`git clean -f`

2.6 Tagging

Command	Description	Example
`git tag`	Creates a tag for a specific commit	`git tag -a v1.0 -m "Version 1.0"`
`git tag -d`	Deletes a tag	`git tag -d v1.0`
`git push --tags`	Pushes tags to a remote repository	`git push origin --tags`

2.7 Advanced

Command	Description	Example
`git bisect`	Finds the commit that introduced a bug	`git bisect start`
`git blame`	Shows which commit and author modified each line of a file	`git blame file.txt`
`git reflog`	Shows a log of changes to the tip of branches	`git reflog`
`git submodule`	Manages external repositories as submodules	`git submodule add https://github.com/user/repo.git`
`git archive`	Creates an archive of the repository files	`git archive --format=zip HEAD > archive.zip`
`git gc`	Cleans up unnecessary files and optimises the repository	`git gc`

2.8 GitHub CLI

Command	Description	Example
`gh auth login`	Logs into GitHub via the command line	`gh auth login`
`gh repo clone`	Clones a GitHub repository	`gh repo clone user/repo`
`gh issue list`	Lists issues in a GitHub repository	`gh issue list`
`gh pr create`	Creates a pull request on GitHub	`gh pr create --title "New Feature" --body "Description"`
`gh repo create`	Creates a new GitHub repository	`gh repo create my-repo`

3. Docker

Docker packages an application and everything it needs into a portable container, so it runs the same way on a laptop, in CI and in production.

3.1 Installation

```
# Windows
winget install Docker.DockerDesktop
# macOS -- drop --cask for the CLI alone, against a remote or Colima daemon
brew install --cask docker
# Linux
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
```

On Linux, the `usermod` line matters. It is what lets you run `docker` without `sudo`, and it only takes effect after you log out and back in. On Windows and macOS, Docker Desktop has to be launched once before the CLI can talk to anything.

Verify: `docker --version` prints the Engine version. `Cannot connect to the Docker daemon` means Docker is installed but not running, which is a different problem.

3.2 Basic

Command	Description	Example
`docker --version`	Displays the installed Docker version	`docker --version`
`docker info`	Shows system-wide information, such as the number of containers and images	`docker info`
`docker pull`	Downloads an image from a registry, Docker Hub by default	`docker pull ubuntu:latest`
`docker images`	Lists all downloaded images	`docker images`
`docker run`	Creates and starts a new container from an image	`docker run -it ubuntu bash`
`docker ps`	Lists running containers	`docker ps`
`docker ps -a`	Lists all containers, including stopped ones	`docker ps -a`
`docker stop`	Stops a running container	`docker stop container_name`
`docker start`	Starts a stopped container	`docker start container_name`
`docker rm`	Removes a container	`docker rm container_name`
`docker rmi`	Removes an image	`docker rmi image_name`
`docker exec`	Runs a command inside a running container	`docker exec -it container_name bash`

3.3 Intermediate

Command	Description	Example
`docker build`	Builds an image from a Dockerfile	`docker build -t my_image .`
`docker commit`	Creates a new image from a container's changes	`docker commit container_name my_image:tag`
`docker logs`	Fetches logs from a container	`docker logs container_name`
`docker inspect`	Returns detailed information about a container or image	`docker inspect container_name`
`docker stats`	Displays live resource usage of running containers	`docker stats`
`docker cp`	Copies files between a container and the host	`docker cp container_name:/path/in/container /path/on/host`
`docker rename`	Renames a container	`docker rename old_name new_name`
`docker network ls`	Lists all Docker networks	`docker network ls`
`docker network create`	Creates a new Docker network	`docker network create my_network`
`docker network inspect`	Shows details about a Docker network	`docker network inspect my_network`
`docker network connect`	Connects a container to a network	`docker network connect my_network container_name`
`docker volume ls`	Lists all Docker volumes	`docker volume ls`
`docker volume create`	Creates a new Docker volume	`docker volume create my_volume`
`docker volume inspect`	Provides details about a volume	`docker volume inspect my_volume`
`docker volume rm`	Removes a Docker volume	`docker volume rm my_volume`

3.4 Advanced

Command	Description	Example
`docker-compose up`	Starts services defined in a `docker-compose.yml` file	`docker-compose up`
`docker-compose down`	Stops and removes services defined in a `docker-compose.yml` file	`docker-compose down`
`docker-compose logs`	Displays logs for services managed by Docker Compose	`docker-compose logs`
`docker-compose exec`	Runs a command in a service's container	`docker-compose exec service_name bash`
`docker save`	Exports an image to a tar file	`docker save -o my_image.tar my_image:tag`
`docker load`	Imports an image from a tar file	`docker load < my_image.tar`
`docker export`	Exports a container's filesystem as a tar file	`docker export container_name > container.tar`

Command	Description	Example
`docker import`	Creates an image from an exported container	`docker import container.tar my_new_image`
`docker system df`	Displays disk usage by Docker objects	`docker system df`
`docker system prune`	Cleans up unused images, containers, volumes and networks	`docker system prune`
`docker tag`	Assigns a new tag to an image	`docker tag old_image_name new_image_name`
`docker push`	Uploads an image to a registry	`docker push my_image:tag`
`docker login`	Logs into a Docker registry	`docker login`
`docker logout`	Logs out of a Docker registry	`docker logout`
`docker swarm init`	Initialises a Docker Swarm mode cluster	`docker swarm init`
`docker service create`	Creates a new service in Swarm mode	`docker service create --name my_service nginx`
`docker stack deploy`	Deploys a stack using a Compose file in Swarm mode	`docker stack deploy -c docker-compose.yml my_stack`
`docker stack rm`	Removes a stack in Swarm mode	`docker stack rm my_stack`
`docker checkpoint create`	Creates a checkpoint for a container	`docker checkpoint create container_name checkpoint_name`
`docker checkpoint ls`	Lists checkpoints for a container	`docker checkpoint ls container_name`
`docker checkpoint rm`	Removes a checkpoint	`docker checkpoint rm container_name checkpoint_name`

4. Kubernetes

Kubernetes is the conductor of a container orchestra. It automates the deployment, scaling and management of containerised applications across a cluster of machines.

4.1 Installation

`kubectl` is the client. It talks to a cluster you already have, so installing it does not give you Kubernetes; for a local cluster use Docker Desktop's built-in one, or `kind`, `k3d` or `minikube`.

```
# Windows
winget install Kubernetes.kubectl
# macOS
brew install kubectl
# Linux -- pinned to the current stable release
curl -LO "https://dl.k8s.io/release/$(curl -Ls
    https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Verify: `kubectl version --client` prints the client version without needing a cluster. Drop `--client` to check the connection as well; `kubectl cluster-info` is the fuller answer.

4.2 Basic

Command	Description	Example
`kubectl version`	Displays the Kubernetes client and server version	`kubectl version --short`
`kubectl cluster-info`	Shows information about the cluster	`kubectl cluster-info`
`kubectl get nodes`	Lists all nodes in the cluster	`kubectl get nodes`
`kubectl get pods`	Lists all pods in the default namespace	`kubectl get pods`
`kubectl get services`	Lists all services in the default namespace	`kubectl get services`
`kubectl get namespaces`	Lists all namespaces in the cluster	`kubectl get namespaces`
`kubectl describe pod`	Shows detailed information about a specific pod	`kubectl describe pod pod-name`
`kubectl logs`	Displays logs for a specific pod	`kubectl logs pod-name`
`kubectl create namespace`	Creates a new namespace	`kubectl create namespace my-namespace`
`kubectl delete pod`	Deletes a specific pod	`kubectl delete pod pod-name`

4.3 Intermediate

Command	Description	Example
`kubectl apply`	Applies changes defined in a YAML file	`kubectl apply -f deployment.yaml`
`kubectl delete`	Deletes resources defined in a YAML file	`kubectl delete -f deployment.yaml`
`kubectl scale`	Scales a deployment to the desired number of replicas	`kubectl scale deployment my-deployment --replicas=3`
`kubectl expose`	Exposes a pod or deployment as a service	`kubectl expose deployment my-deployment --type=LoadBalancer --port=80`
`kubectl exec`	Executes a command in a running pod	`kubectl exec -it pod-name -- /bin/bash`
`kubectl port-forward`	Forwards a local port to a port in a pod	`kubectl port-forward pod-name 8080:80`
`kubectl get configmaps`	Lists all ConfigMaps in the namespace	`kubectl get configmaps`
`kubectl get secrets`	Lists all Secrets in the namespace	`kubectl get secrets`
`kubectl edit`	Edits a resource definition directly in the editor	`kubectl edit deployment my-deployment`
`kubectl rollout status`	Displays the status of a deployment rollout	`kubectl rollout status deployment/my-deployment`

4.4 Advanced

Command	Description	Example
`kubectl rollout undo`	Rolls back a deployment to a previous revision	`kubectl rollout undo deployment/my-deployment`
`kubectl top nodes`	Shows resource usage for nodes	`kubectl top nodes`
`kubectl top pods`	Displays resource usage for pods	`kubectl top pods`
`kubectl cordon`	Marks a node as unschedulable	`kubectl cordon node-name`
`kubectl uncordon`	Marks a node as schedulable	`kubectl uncordon node-name`
`kubectl drain`	Safely evicts all pods from a node	`kubectl drain node-name --ignore-daemonsets`
`kubectl taint`	Adds a taint to a node to control pod placement	`kubectl taint nodes node-name key=value:NoSchedule`
`kubectl get events`	Lists all events in the cluster	`kubectl get events`
`kubectl apply -k`	Applies resources from a kustomization directory	`kubectl apply -k ./kustomization-dir/`
`kubectl config view`	Displays the kubeconfig file	`kubectl config view`
`kubectl config use-context`	Switches the active context in kubeconfig	`kubectl config use-context my-cluster`
`kubectl debug`	Creates a debugging session for a pod	`kubectl debug pod-name`

Command	Description	Example
`kubectl delete namespace`	Deletes a namespace and its resources	`kubectl delete namespace my-namespace`
`kubectl patch`	Updates a resource using a patch	`kubectl patch deployment my-deployment -p '{"spec": {"replicas": 2}}'`
`kubectl rollout history`	Shows the rollout history of a deployment	`kubectl rollout history deployment my-deployment`
`kubectl autoscale`	Automatically scales a deployment based on resource usage	`kubectl autoscale deployment my-deployment --cpu-percent=50 --min=1 --max=10`
`kubectl label`	Adds or modifies a label on a resource	`kubectl label pod pod-name environment=production`
`kubectl annotate`	Adds or modifies an annotation on a resource	`kubectl annotate pod pod-name description="My app pod"`
`kubectl delete pv`	Deletes a PersistentVolume	`kubectl delete pv my-pv`
`kubectl get ingress`	Lists all Ingress resources in the namespace	`kubectl get ingress`
`kubectl create configmap`	Creates a ConfigMap from a file or literal values	`kubectl create configmap my-config --from-literal=key1=value1`
`kubectl create secret`	Creates a Secret from a file or literal values	`kubectl create secret generic my-secret --from-literal=password=myPassword`
`kubectl api-resources`	Lists all available API resources in the cluster	`kubectl api-resources`
`kubectl api-versions`	Lists all API versions supported by the cluster	`kubectl api-versions`
`kubectl get crds`	Lists all CustomResourceDefinitions	`kubectl get crds`

5. Helm

Helm is the package manager for Kubernetes. It installs and manages applications packaged as charts, the way `apt` installs packages on Debian.

5.1 Installation

```
# Windows
winget install Helm.Helm
# macOS
brew install helm
# Linux -- the official installer script
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
```

Read that script before running it on a machine that matters: it installs a binary into `/usr/local/bin` with `sudo`. If your distribution has snapd, `sudo snap install helm --classic` does the same job.

Verify: `helm version` prints `version.BuildInfo`. Helm 3 has no server component, but it reads your kubeconfig, so if it fails on a connection error the thing to fix is `kubectl`, not Helm.

5.2 Basic

Command	Description	Example
`helm help`	Displays help for the Helm CLI or a specific command	`helm help`
`helm version`	Shows the Helm client and server version	`helm version`
`helm repo add`	Adds a new chart repository	`helm repo add stable https://charts.helm.sh/stable`
`helm repo update`	Updates all chart repositories to the latest version	`helm repo update`
`helm repo list`	Lists all the repositories added to Helm	`helm repo list`
`helm search hub`	Searches for charts on Helm Hub	`helm search hub nginx`
`helm search repo`	Searches for charts in the repositories	`helm search repo stable/nginx`
`helm show chart`	Displays a chart's metadata and dependencies	`helm show chart stable/nginx`

5.3 Installing and upgrading charts

Command	Description	Example
`helm install`	Installs a chart into a Kubernetes cluster	`helm install my-release stable/nginx`
`helm upgrade`	Upgrades an existing release with a new version of the chart	`helm upgrade my-release stable/nginx`
`helm upgrade --install`	Installs a chart if it is not installed, upgrades it if it is	`helm upgrade --install my-release stable/nginx`
`helm uninstall`	Uninstalls a release	`helm uninstall my-release`
`helm list`	Lists all the releases installed on the cluster	`helm list`
`helm status`	Displays the status of a release	`helm status my-release`

5.4 Working with charts

Command	Description	Example
`helm create`	Creates a new Helm chart in a specified directory	`helm create my-chart`
`helm lint`	Lints a chart to check for common errors	`helm lint ./my-chart`
`helm package`	Packages a chart into a `.tgz` file	`helm package ./my-chart`
`helm template`	Renders the Kubernetes YAML from a chart without installing it	`helm template my-release ./my-chart`
`helm dependency update`	Updates the dependencies in `Chart.yaml`	`helm dependency update ./my-chart`
`helm dependency build`	Builds dependencies for a chart	`helm dependency build ./my-chart`
`helm dependency list`	Lists all dependencies for a chart	`helm dependency list ./my-chart`

5.5 Inspecting and testing

Command	Description	Example
`helm get all`	Gets all information for a release, values and templates included	`helm get all my-release`
`helm get values`	Displays the values used in a release	`helm get values my-release`
`helm test`	Runs the tests defined in a chart	`helm test my-release`
`helm template --debug`	Renders manifests and includes debug output	`helm template my-release ./my-chart --debug`
`helm install --dry-run`	Simulates an installation without performing it	`helm install my-release stable/nginx --dry-run`

Command	Description	Example
`helm upgrade --dry-run`	Simulates an upgrade without applying it	`helm upgrade my-release stable/nginx --dry-run`

5.6 Values and customisation

Command	Description	Example
`helm install --values`	Installs a chart with custom values	`helm install my-release stable/nginx --values values.yaml`
`helm upgrade --values`	Upgrades a release with custom values	`helm upgrade my-release stable/nginx --values values.yaml`
`helm install --set`	Installs a chart with a value set on the command line	`helm install my-release stable/nginx --set replicaCount=3`
`helm upgrade --set`	Upgrades a release with a value set on the command line	`helm upgrade my-release stable/nginx --set replicaCount=5`
`helm uninstall --purge`	Removes a release and its resources, release history included	`helm uninstall my-release --purge`

5.7 Namespaces and contexts

Command	Description	Example
`helm list --namespace`	Lists releases in a specific namespace	`helm list --namespace kube-system`
`helm list -n`	Short form of the same thing	`helm list -n kube-system`
`helm install --namespace`	Installs a chart into a specific namespace	`helm install my-release stable/nginx --namespace mynamespace`
`helm upgrade --namespace`	Upgrades a release in a specific namespace	`helm upgrade my-release stable/nginx --namespace mynamespace`
`helm uninstall --namespace`	Uninstalls a release from a specific namespace	`helm uninstall my-release --namespace kube-system`
`helm install --kube-context`	Installs a chart into the cluster named by a kubeconfig context	`helm install my-release stable/nginx --kube-context my-cluster`
`helm upgrade --kube-context`	Upgrades a release in a specific context	`helm upgrade my-release stable/nginx --kube-context my-cluster`

5.8 Repositories and publishing

Command	Description	Example
`helm repo remove`	Removes a chart repository	`helm repo remove stable`
`helm repo index`	Creates or updates the index file for a chart repository	`helm repo index ./charts`
`helm package --sign`	Packages a chart and signs it with a GPG key	`helm package ./my-chart --sign --key my-key-id`
`helm create --starter`	Creates a new chart from a starter template	`helm create --starter https://github.com/helm/charts.git`
`helm push`	Pushes a chart to a chart repository	`helm push ./my-chart my-repo`

5.9 History and rollbacks

Command	Description	Example
`helm rollback`	Rolls a release back to a previous version	`helm rollback my-release 1`
`helm history`	Displays the history of a release	`helm history my-release`
`helm history --max`	Limits the number of versions shown in the history	`helm history my-release --max 5`
`helm rollback --recreate-pods`	Rolls back to a previous version and recreates the pods	`helm rollback my-release 2 --recreate-pods`

6. Terraform

Terraform builds cloud infrastructure from code. Instead of clicking through an AWS, GCP or Azure console, servers and services are declared in configuration files and Terraform reconciles reality with them.

6.1 Installation

```
# Windows
winget install HashiCorp.Terraform
# macOS
brew install hashicorp/tap/terraform
# Linux (Debian/Ubuntu) -- the HashiCorp apt repository
wget -O- https://apt.releases.hashicorp.com/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

On the RHEL family use `dnf config-manager` against `https://rpm.releases.hashicorp.com` instead.

Verify: `terraform -version` prints the version and warns when a newer one is out. Terraform needs no daemon, but every provider it uses is downloaded on `terraform init`, so that is the command that first needs network access.

6.2 Commands

Command	Description
`terraform --help`	Display general help for the Terraform CLI
`terraform init`	Initialise the working directory and download the provider plugins the configuration needs
`terraform validate`	Check the configuration files for syntax errors and internal inconsistencies
`terraform plan`	Produce an execution plan showing what Terraform will do to reach the desired state
`terraform apply`	Apply the changes required to reach the desired state, prompting for approval first
`terraform show`	Display the state, or a saved plan, in human-readable form
`terraform output`	Display the output values defined in the configuration
`terraform destroy`	Destroy the infrastructure the configuration manages, prompting for confirmation
`terraform refresh`	Update the state file from the real infrastructure without applying any change
`terraform taint`	Mark a resource for recreation on the next apply, even if nothing about it changed
`terraform untaint`	Remove the tainted status from a resource
`terraform state`	Manage the state file: move resources between modules, remove them, inspect them

Command	Description
`terraform state list`	List every resource tracked in the state file
`terraform import`	Bring existing infrastructure under Terraform management
`terraform graph`	Generate a graph of the resources and their dependencies
`terraform providers`	List the providers the current configuration requires

On the backend: The backend is configured in a `backend` block inside `terraform { }`, not from the command line. It decides where the state file lives: locally by default, or remotely in S3, Azure Blob Storage, Terraform Cloud and others. `terraform init` is what reads it and migrates existing state.